



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

An Intelligent AI Coding Mentor for Guided Programming

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1707-Artificial Intelligence

to the award of the degree of

BACHELOR OF TECHNOLOGY IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Submitted by

Vasanth Nithi D Y (192424137)

Under the Supervision of Dr.

Veeramani R

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**An Intelligent AI Coding Mentor for Guided Programming**” has been carried out by **Vasanth Nithi D Y (192424137)** under the supervision of **Veeramani R** and is submitted in partial fulfilment of the requirements for the current semester of the **B.Tech Artificial Intelligence and Data Science** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr Krishna Kumar Kathiresan,
Professor,
Generative AI,
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr Veeramani R,
Professor,
Quantum Intelligence,
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, **Vasantha Nithi D Y** of the B.Tech Artificial Intelligence and Data Science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**An Intelligent AI Coding Mentor for Guided Programming**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsi bilities	Design & Development Contribution	Testing & Analysis Contributio n	Report Contributio n	Approx. Contribution (%)
Vasanth Nithi D Y 192424137	Requireme nt analysis, literature review and system planning.	Designed the AI mentor architecture, code-analysis workflow and learning- oriented feedback mechanism.	Prepared evaluation criteria, test cases and performanc e-analysis framework.	Prepared the methodology, design analysis and consolidated the final report.	35%
MUGESH R (192524454)	Frontend developme nt, interface design and system integration	Developed the user interface, interaction workflow and application components.	Conducted functional testing and analysed system behaviour.	Prepared implementation and testing- related documentation.	35%
Pritham G (192572014)	Backend developme nt, AI integration and data manageme nt.	Implemented backend services, AI integration and code-processing components.	Performed performance testing, result analysis and validation.	Prepared results, conclusion, future work and supporting documentation.	30%
Total					100%

ABSTRACT

Programming education is challenging for beginners because learning to write code requires the simultaneous development of conceptual understanding, syntax knowledge, debugging ability and problem-solving skills. Conventional coding platforms can provide exercises and correctness feedback, while modern generative AI tools can explain or generate code; however, these approaches can leave a gap between receiving an answer and learning how to arrive at that answer independently. This project proposes an Intelligent AI Coding Mentor for Guided Programming, an educational system intended to provide contextual and progressive assistance rather than immediately replacing the learner's reasoning process. The proposed system combines deterministic code analysis with an AI-based mentoring layer. Submitted code is examined for common syntax, runtime and logical issues, after which structured information is passed to the mentor component to produce explanations, hints and learning-oriented guidance. A learner-profile component can record performance patterns and support adaptive exercise recommendation. The design also incorporates secure code execution, privacy considerations, response-time constraints and safeguards against excessive dependence on generated solutions. The project follows an iterative engineering approach covering requirements analysis, system design, interface development, code analysis, AI integration, learner modelling, testing and evaluation. The evaluation framework measures error-detection performance, explanation quality, hint usefulness, recommendation suitability, response time, error-resolution rate and learner satisfaction. Importantly, the report distinguishes between the proposed evaluation framework and measured project results so that no performance value is presented without experimental evidence. The expected contribution is a practical learning-oriented AI mentor architecture that combines reliable program analysis with flexible natural-language assistance while keeping independent problem solving at the centre of the learning experience.

Keywords: Artificial Intelligence, Programming Education, Intelligent Tutoring Systems, Code Analysis, Large Language Models, Adaptive Learning

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	10
2	CHAPTER 2: Literature Review	14
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	17
4	CHAPTER 4: Implementation, Testing & Results, Project Management	21
5	CHAPTER 5: Conclusion & Reflection	26
	References	28
	Appendices	30

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.	Architecture Diagram	19

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Programming is an essential skill in computer science, artificial intelligence, data science, and other engineering disciplines. However, beginner programmers often face difficulties in understanding programming concepts, identifying errors, developing logical thinking, and converting problem statements into working programs.

Traditional programming education commonly depends on lectures, textbooks, online tutorials, and predefined programming exercises. Although these resources provide useful information, they may not provide personalized, real-time guidance according to the learner's individual knowledge level and mistakes.

The proposed Intelligent AI Coding Mentor for Guided Programming Learning is an AI-assisted educational system designed to support students while learning programming. The system acts as a virtual coding mentor by analysing a learner's code, identifying programming errors, providing explanations, generating hints, recommending appropriate learning resources, and gradually guiding the learner toward a solution.

Instead of simply providing the final answer, the proposed system emphasizes guided learning. It encourages learners to understand the underlying programming concept and solve problems independently.

The system can combine natural language processing, code analysis, error detection, learner profiling, and adaptive recommendation techniques to provide a personalized programming-learning environment.

1.2 Need for the Project

The project is required because many beginner programmers experience the following difficulties:

- Difficulty understanding programming fundamentals.
- Lack of immediate feedback while writing code.
- Difficulty identifying syntax and logical errors.
- Dependence on copying solutions from online sources.
- Difficulty selecting suitable programming exercises.

- Lack of personalized learning paths.
- Limited access to one-to-one programming mentors.
- Difficulty understanding compiler or interpreter error messages.

Existing programming platforms generally provide exercises, automated evaluation, or coding assistance, but a learner may still require an explanation of why an error occurred and how to correct it.

Therefore, an intelligent mentor that combines coding assistance with educational guidance can improve the learning experience.

Engineering Significance

The project combines:

- Artificial Intelligence
- Natural Language Processing
- Source-code analysis
- Machine learning
- Educational technology
- Personalized recommendation
- Human-computer interaction

The system therefore represents an interdisciplinary engineering solution for improving programming education

1.3 Problem Statement

Beginner programmers frequently struggle to understand programming concepts, identify errors, and develop problem-solving skills because conventional programming-learning systems often provide limited personalization and immediate contextual guidance.

The engineering problem is to design and develop an intelligent AI-based coding mentor that can analyse learner code, identify errors, understand the learner's difficulty level, provide appropriate hints and explanations, and recommend progressively suitable programming tasks while avoiding excessive dependence on automatically generated solutions.

The system must balance accuracy, educational usefulness, response time, personalization, privacy, security, and computational cost.

1.4 Project Aim

The main aim of this project is:

To design and develop an intelligent AI coding mentor that provides personalized, step-by-step guidance to learners while they practice programming, helping them understand errors, improve problem-solving skills, and progressively develop coding proficiency.

1.5 Project Objectives

1. To design an AI-based platform for guided programming learning.
2. To develop a code-analysis module capable of identifying common syntax, runtime, and logical errors.
3. To provide contextual explanations of programming errors using natural language.
4. To generate progressive hints rather than immediately revealing complete solutions.
5. To develop a learner profile based on programming performance and learning progress.
6. To recommend programming exercises according to the learner's skill level.
7. To evaluate the effectiveness of the AI mentor using predefined testing metrics.
8. To compare the proposed system with conventional coding-learning approaches.

1.6 Scope

Included

The project includes:

- Programming learning assistance.
- Code submission and analysis.
- Error identification.
- AI-generated explanations.
- Progressive hints.
- Programming-question recommendation.
- Learner progress tracking.
- Difficulty-level adaptation.
- Basic performance analytics.
- Interactive AI mentor interface.

The project does not primarily focus on:

- Fully autonomous programming education without human involvement.
- Professional software development automation.
- Replacing teachers or instructors.

- Supporting every programming language.
- High-risk production software deployment.

Assumptions

- The learner has basic computer knowledge.
- The learner can submit programming code.
- The system has access to a suitable AI model or inference service.
- Programming exercises have predefined concepts and difficulty levels.

Boundary Conditions

The system's recommendations depend on the quality of the learner's submitted code, exercise metadata, AI model capabilities, and available computational resources.

1.7 Expected Engineering Outcome

The expected outcome is a functional AI Coding Mentor prototype containing:

- User interface.
- Code editor.
- Code analysis module.
- AI mentor module.
- Error explanation system.
- Hint-generation mechanism.
- Learning recommendation module.
- Progress tracking module.
- Database/user-profile component.

The overall system should provide learners with personalized programming guidance and measurable learning feedback

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review should examine research and existing technologies related to:

- AI-assisted programming education.
- Intelligent tutoring systems.
- Automated programming assessment.
- AI code-generation systems.
- Code debugging assistants.
- Adaptive learning systems.
- Natural language-based educational systems.
- Personalized recommendation systems.

The review should compare their functionality, educational approach, advantages, and limitations.

2.2 Review of Existing Work

1. Traditional Online Coding Platforms

Traditional coding platforms provide programming problems, coding environments, test cases, and automated evaluation.

Advantages:

- Large exercise collections.
- Automatic evaluation.
- Immediate correctness feedback.
- Structured programming practice.

Limitations:

- Limited personalized explanations.
- Feedback may focus primarily on correctness.
- Beginners may not understand why their solution failed.

2. Intelligent Tutoring Systems

Intelligent tutoring systems attempt to provide personalized educational support based on learner performance.

Advantages:

- Adaptive learning.
- Personalized feedback.
- Learner modelling.

Limitations:

- Some systems are limited to particular subjects.
- Advanced implementation can be computationally complex.

3. AI Coding Assistants

AI coding assistants can generate code, explain code, detect errors, and suggest modifications.

Advantages:

- Fast feedback.
- Natural-language interaction.
- Code explanation.
- Debugging support.

Limitations:

- Can provide complete solutions too quickly.
- Generated code may contain errors.
- Excessive dependence can reduce independent problem solving.

4. Proposed AI Coding Mentor

The proposed system focuses specifically on guided programming education.

Instead of simply generating the solution, it can follow a staged approach:

Error → Explanation → Hint → Concept → Guided Correction → Verification

This makes the system more suitable for learning than a purely code-generation-oriented assistant.

2.3 Comparative Analysis

Existing Approach		Advantages	Limitations	Relevance to Proposed Work
Traditional Classes	Coding	Human guidance	Limited individual attention	Provides educational foundation
Online Platforms	Coding	Automated evaluation	Limited personalization	Exercise and testing component
Intelligent Systems	Tutoring	Adaptive learning	More complex implementation	Learner modelling
AI Coding Assistants		Fast code assistance	May encourage copying	AI interaction
Proposed AI Coding Mentor		Personalized learning	guided AI accuracy and constraints	Combines learning + AI + coding

2.4 Research / Engineering Gap

A significant gap exists between AI code generation and AI-guided programming education.

Many AI coding tools prioritize producing or correcting code. However, beginner learners require more than a correct program. They need to understand:

- What went wrong?
- Why did it go wrong?
- Which concept is missing?
- What should they try next?
- How can they solve a similar problem independently?

The proposed system addresses this gap by prioritizing learning-oriented interaction rather than answer generation.

2.5 Proposed Contribution

The project proposes an AI coding mentor that integrates:

1. Code analysis.
2. Error classification.
3. Natural-language explanation.
4. Progressive hints.
5. Learner profiling.
6. Adaptive exercise recommendation.
7. Learning-progress tracking.

The key contribution is the integration of these components into a single guided-learning workflow.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student	Easy-to-use coding interface
Student	Immediate error explanation
Student	Progressive hints
Student	Personalized exercises
Faculty	Learning progress information
Administrator	Secure user management
Institution	Reliable and maintainable system

Functional Requirements

Requirement	Type	Measurable Target
Code submission	Functional	Successful submission
Error detection	Functional	Correctly identify common errors
AI explanation	Functional	Generate understandable feedback
Hint generation	Functional	Provide progressive hints
Exercise recommendation	Functional	Recommend suitable difficulty
Progress tracking	Functional	Store learner performance
User authentication	Functional	Secure login

Non-Functional Requirements

Requirement	Type	Target
Response time	Non-functional	Low-latency feedback
Usability	Non-functional	Beginner-friendly interface
Reliability	Non-functional	Stable operation
Security	Non-functional	Protect learner data
Scalability	Non-functional	Support increasing users

Requirement	Type	Target
Maintainability	Non-functional	Modular architecture

3.2 Constraints & Applicable Standards

Relevant considerations include:

- Data privacy.
- Secure authentication.
- Responsible AI usage.
- Protection of student information.
- Secure code execution.
- Prevention of malicious code execution.
- Model reliability.
- Computational resource limitations.

Particular attention should be given to sandboxing submitted programs, because executing untrusted code can create security risks.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Code analysis	Accurate common-error detection	%	High
AI response	Understandable feedback	Rating	High
Response time	Low latency	seconds	High
Hint system	Progressive hints	levels	High
Recommendation	Appropriate difficulty	level	High
Availability	Reliable system operation	%	Medium
Security	Protected code execution	qualitative	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Rule-Based Coding Mentor

Uses predefined rules to detect common programming errors and provide fixed explanations.

Alternative 2: Generative AI Coding Assistant

Uses a large language model to analyse code and generate explanations and solutions.

Alternative 3: Hybrid AI Coding Mentor

Combines deterministic code analysis with an AI model and learner-profile system.

Criterion	Weight	Rule-Based	Generative AI	Hybrid AI
Performance	25%	3	4	5
Cost	15%	5	3	4
Educational Guidance	25%	3	4	5
Safety	15%	5	3	4
Personalization	20%	2	4	5

The hybrid architecture provides a balance between deterministic programming analysis and flexible natural-language AI guidance.

Proposed System Architecture

The architecture separates code execution, code analysis, AI reasoning, and learner modelling. This improves modularity and allows individual components to be upgraded independently.



Fig 1. Architecture Diagram

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision
AI Model	Rule-based system	Flexible explanations	Higher computational cost	AI-assisted
Code Analysis	AI only	Simple architecture	Less deterministic	Hybrid
Feedback	Full solution	Fast completion	Reduces learning	Progressive hints
Execution	Direct execution	Simple	Security risk	Sandboxed execution

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational resource usage	AI inference requirements	Optimize prompts/model usage
Ethics	Learner dependence on AI	AI-assisted learning risks	Use hints before solutions
Health & Safety	Screen-learning workload	User interaction	Simple interface
Security	Malicious code execution	Untrusted submissions	Sandbox execution
Privacy	Learner performance data	User profiles	Minimize and protect stored data

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect AI explanation	Medium	High	High	Validate responses	Medium
AI-generated wrong code	Medium	Medium	Medium	Code testing	Low
Malicious code execution	Medium	High	High	Sandbox	Low
Data leakage	Low	High	Medium	Authentication/encryption	Low
Excessive AI dependence	Medium	Medium	Medium	Hint-based learning	Low
High API/inference cost	Medium	Medium	Medium	Optimize requests	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

An iterative development approach can be used.

Development stages

1. Requirement analysis.
2. System architecture design.
3. User-interface development.
4. Code-analysis implementation.
5. AI mentor integration.
6. Learner-profile development.
7. Recommendation system implementation.
8. Testing.
9. Performance evaluation.
10. Final integration.

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
Python	Backend and AI implementation	Development
HTML/CSS/JavaScript	User interface	Development
Python code-analysis libraries	Program analysis	Development
AI/LLM API or local model	Mentor responses	AI integration
Database	User and progress storage	Development
Git	Version control	All stages
VS Code	Development environment	All stages

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Code submission	Submit valid program	Program accepted
Syntax detection	Submit syntax error	Error identified
Runtime detection	Execute faulty program	Runtime error identified
Explanation	Request explanation	Clear explanation produced
Hint generation	Request hint	Hint provided without immediate answer
Recommendation	Complete exercises	Appropriate next task recommended
Progress tracking	Complete exercise	Score/progress recorded
Security	Submit unsafe input	Execution safely restricted

Testing Specification

Specification	Required Value	Achieved Value	Status
Code submission	100% successful	To be measured	—
Error detection	Target accuracy	To be measured	—
AI explanation	Acceptable quality	To be measured	—
Recommendation	Suitable difficulty	To be measured	—
Response time	Target limit	To be measured	—

Important: Actual achieved values should be filled using your project's real test results rather than invented values.

4.4 Results

The following results can be measured:

- Error detection accuracy.
- AI explanation quality.
- Hint usefulness.
- Exercise recommendation accuracy.
- Average response time.
- User satisfaction.
- Programming improvement before and after using the mentor.

Suggested Result Graphs

1. Error Detection Accuracy
2. Average AI Response Time
3. Learner Performance Before vs After
4. Hint Usage vs Successful Attempts
5. User Satisfaction
6. Exercise Recommendation Accuracy

4.5 Data Analysis & Engineering Judgment

The collected test data should be analysed to determine whether the AI mentor improves programming-learning performance.

For example, learner performance can be measured before and after using the system.

A useful metric is:

Improvement (%) =

$$\frac{\text{Post-test Score} - \text{Pre-test Score}}{\text{Pre-test Score}} \times 100$$

Additional metrics can include:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Average response time.
- Error-resolution rate.
- Hint-to-solution conversion rate.
- User satisfaction score.

The engineering judgement should consider not only technical accuracy but also whether the system actually promotes independent learning.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop AI coding mentor	Working prototype	Fully/Partially
Detect programming errors	Test results	Fully/Partially
Provide explanations	AI feedback tests	Fully/Partially
Provide progressive hints	Hint module	Fully/Partially
Personalize learning	Learner model	Fully/Partially
Recommend exercises	Recommendation module	Fully/Partially

Objective	Evidence	Achievement
Track learner progress	Database/dashboard	Fully/Partially

Again, mark Fully/Partially/Not Achieved based on your actual implementation.

4.7 Strengths & Limitations

Strengths

- Personalized programming assistance.
- Immediate feedback.
- Beginner-friendly explanations.
- Progressive hint mechanism.
- Adaptive exercise recommendations.
- Reduces dependence on searching for debugging solutions.
- Supports self-paced learning.
- Provides continuous learning feedback.

Limitations

- AI responses may occasionally be incorrect.
- AI inference can require computational resources.
- The system may not support every programming language.
- Learners may become overly dependent on AI.
- Code analysis may not identify every logical error.
- Recommendation quality depends on learner data.

4.8 Project Planning, Roles & Budget

Suggested Milestones

Phase	Activity
Week 1–2	Problem identification & literature review
Week 3–4	Requirement analysis
Week 5–6	System architecture
Week 7–9	UI and backend development
Week 10–12	AI mentor integration
Week 13–14	Testing
Week 15	Data analysis
Week 16	Final documentation and presentation

Budget

Item	Estimated Cost
Development software	₹0

Item	Estimated Cost
Open-source libraries	₹0
Cloud/API usage	₹1,000–₹3,000
Hosting	₹0–₹1,000
Testing resources	₹500
Miscellaneous	₹500
Estimated Total	₹2,000–₹5,000

The actual budget should be replaced with your project's real expenditure.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Intelligent AI Coding Mentor for Guided Programming Learning is designed to address common difficulties faced by beginner programmers during programming education. The proposed system combines AI-based assistance, code analysis, error explanation, progressive hints, learner modelling, and adaptive exercise recommendation into a unified learning platform. Unlike a conventional coding assistant that primarily focuses on generating solutions, the proposed system emphasizes guided learning and independent problem solving. The mentor can identify programming difficulties, explain errors in simple language, provide appropriate hints, and recommend exercises according to learner progress. The project therefore provides a potential engineering solution for delivering personalized and interactive programming education.

5.2 Future Work

Future improvements may include:

- Support for multiple programming languages.
- Voice-based AI programming assistance.
- Advanced learner behaviour analysis.
- Personalized learning paths.
- Integration with college Learning Management Systems.
- Mobile application development.
- Advanced code-quality analysis.
- AI-generated programming quizzes.
- Gamification and achievement systems.
- Offline/local AI model support.
- Faculty dashboards and analytics.
- More advanced adaptive learning algorithms.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Artificial intelligence integration.
- Natural language processing.
- Source-code analysis.
- AI prompt design.
- Web application development.
- Database management.
- Software testing.
- Responsible AI principles.

Engineering & Professional Skills Developed

- Problem analysis.
- System architecture design.
- Programming.
- Debugging.
- Software testing.
- Technical documentation.
- Team collaboration.
- Project management.

Individual Reflection

Most difficult engineering problem encountered: The most difficult problem was designing an AI feedback mechanism that could guide learners without directly giving the complete solution. This required balancing AI assistance with the educational objective of encouraging independent problem solving.

Key engineering decision: A hybrid approach was selected because deterministic code analysis can provide reliable technical information while AI can convert that information into understandable explanations and hints.

New knowledge acquired independently: The project provided practical knowledge about AI-assisted programming, prompt design, code analysis, learner modelling, and secure execution of user-submitted code.

What would be redesigned if repeated: If the project were repeated, the learner modelling and evaluation framework would be designed earlier so that personalization and learning improvement could be measured more systematically.

REFERENCES

1. Crow, T., Luxton-Reilly, A., & Wünsche, B. – “Intelligent Tutoring Systems for Programming Education: A Systematic Review.”
ACM, 2018.
[Research Paper – ACM / DOI](#)
2. “Enhancing Python Learning with PyTutor: Efficacy of a ChatGPT-Based Intelligent Tutoring System in Programming Education.”
Computers and Education: Artificial Intelligence, 2024.
[Research Paper – ScienceDirect](#)
3. “Artificial Intelligence in Computer Programming Education: A Systematic Literature Review.”
Computers and Education: Artificial Intelligence, 2025.
[Research Paper – ScienceDirect](#)
4. “Use of AI-Driven Code Generation Models in Teaching and Learning Programming: A Systematic Literature Review.”
ACM SIGCSE, 2024.
[Research Paper – TU Delft Research Portal](#)
5. “An Intelligent Tutoring System for Programming Education Based on Informative Tutoring Feedback.”
Interactive Technology and Smart Education, 2025.
Research Paper – ERIC
6. “How Does LLM-Powered Coding Assistance Shape Incidental Learning?”
Proceedings of the AAAI Conference on Artificial Intelligence, 2026.
[Research Paper – AAAI](#)
7. “Saksham AI: Advancing AI-Assisted Coding Education for Engineering Students in India Through Socratic Tutoring and Comprehensive Feedback.”
ACM Journal on Computing and Sustainable Societies, 2026.
[Research Paper – ACM DOI](#)
8. “Systematic Review of Large Language Model Applications in Programming Education.”
2026.
Research Paper – SAGE
9. CodeQL Documentation – Code Analysis and Security.
GitHub CodeQL documentation can support the code-analysis/security component of your proposed system.
[Official CodeQL Documentation](#)

10. GitHub CodeQL – Open Source Repository.

Useful as a technical reference for automated source-code analysis.

[Official GitHub CodeQL Repository](#)

APPENDICES

Following your reference report's structure, the appendices can contain:

Appendix A – Detailed Calculations

- Accuracy calculations.
- Performance metrics.
- Statistical analysis.

Appendix B – System Architecture / Flowcharts

- System architecture.
- Data-flow diagram.
- AI mentor workflow.
- User interaction flowchart.

Appendix C – Source Code / Repository Information

- Important code excerpts.
- Repository information.
- API integration details.

Appendix D – Software Documentation

- AI model details.
- Libraries.
- Frameworks.

Appendix E – Experimental / Raw Data

- Test cases.
- User evaluation results.
- Performance measurements.

Appendix F – Ethics / Institutional Approval

- Consent forms or approval documents, where applicable.

Appendix G – Project Meeting / Progress Records

Appendix H – User Feedback

Appendix I – Publications / Competitions / Product Outcomes

Appendix J – Individual Contribution Evidence

These appendix categories follow the structure specified in your reference report.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
•		
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole Report + Viva
Ethics and professional responsibility	SO4	Ch. 3.7
Teamwork and leadership	SO5	Ch. 4.8
Experimentation and engineering judgment	SO6	Ch. 4.3–4.6
Independent acquisition of knowledge	SO7	Ch. 5.3
Standards	SO2	Ch. 3.2
Sustainability and societal impact	SO2/SO4	Ch. 3.7
Risk assessment and mitigation	SO2/SO4	Ch. 3.7
Security considerations	Relevant SO	Ch. 3.7
Integrated/system-level engineering	SO1/SO2	Ch. 3.5
Project planning and management	SO5	Ch. 4.8
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4.8